

What is the difference between scanning and foot printing? With scanning, hackers start to interact with the target system, exchanging packets. It's not completely stealthy. Why is it needed? The information the attacker has collected in the first phase is not complete.

For example, with IP addresses collected from international registries, he only has a limited view, given by a range of IP addresses. Some of these IPs, however, won't be used, but as of now the attacker has no knowledge about this. To actually obtain this information, the attacker needs to interact with the system.

The objective of scanning (port, system, network scanning) is to establish if a given IP address is alive, so that there is a machine at that IP that is running. Also, to determine which services are running on that machine. Another thing is the running operating system is running on that host.

Let's start with the first part: determining if a system is alive. We may do it simply by sending a message like a ping. Also, it should not be easily detectable. The technique is the network ping sweep.

The first approach is ARP host discovery. The ARP protocol is used to discover if a node is alive and running. It's the easiest and the fastest approach. It works by first starting an arp-scan, then running nmap with different options to discover the services available. It can be run with arp-scan command. With Cain, used for Windows, we can go beyond host and service discovery.

Second protocol that can be used to determine if the system is alive is the ICMP protocol. It's a less stealthier approach. It's specifically designed to check if an host is alive. The native tool on linux is ping, implemented using the ICMP protocol.

The attacker will send an ICMP echo request, and it will get back an echo reply. There are other ways to perform this, like with an ICMP timestamp request, or ICMP address mask. nmap is also useful here, as it can be setup to do this ICMP pings. But there are other tools as well, like Hping3. The main advantage is that ICMP protocol can be started outside the network, unlike ARP. We may ask ourselves now, why is this protocol allowed by firewalls? We may find three main situations in organisations:

- Organisations with high security concerns, in which the protocol is not allow from connections from outside, but also for the inside ICMP.

- The border firewall won't accept any of these packets, but it can be used on the inside.
- ICMP is allowed inside, and there are specific IP addresses from which this protocol is accepted, usually other IP addresses of the same ISP.

The last approach is based on the use of TCP and UDP. It's the slowest and the most detectable. This has to be used when internal and external ICMP is not permitted. It will always work, as alive systems need to provide TCP or UDP services. Even by having a local firewall, we still need some services to be activated.

`nmap` does not only use ICMP traffic, as to be more accurate it also uses arp-pings, tcp-pings, etc.

What are the counter measures to these kind of ping sweeps? The first one is an intrusion detection system, which are already trained to detect this type of traffic, in particular for TCP/UDP traffic. Another one is given by the use of firewalls. The major problem is that typically there is not a lot of human attention, while it should be necessary as well. But why is it necessary? It's important because this is one of the situations in which the defender has an advantage in respect to the attacker, having full control of the target system. Once the attacker is inside, the information the defender might obtain could be misleading.

Today, these scans are automatic, performed by tools. Analysing all of that could be problematic, as in a major organisation scans could happen a lot of times during one hour. So, for prevention, tools can be used as well, like ACL in firewalls.

*There is a SANS institute. It's a research institute. It offers specialised training for IT professionals. They also produce a lot of free papers, like "ICMP attacks illustrated".*

After finding out which hosts are running, we need to find out which services are running on given hosts. We will need to determine TCP/UDP services.

What are the scan types?

- TCP connect scan (3-way handshake). *If you want, take a look once again at how the TCP handshake works.* What is the problem with this approach? It's super slow, as we need to exchange three packets. Also, it's quite easy to detect. Remember that TCP handshake establishes a connection, but the attacker does not want to do this. So, he just sends a `syn` packet, and if receives a `syn+ack` he will know that the port is open, without the need of

sending an ACK. There are a lot of variations, which are all ways to trick the handshake, without using the full protocol.

- UDP scans: we don't know if the service is open or not until we receive UDP packets back, but for the nature of UDP the packet might arrive whenever. So, tools set a timeout of up to 1-2 seconds, and if nothing is received they assume the port is closed.

All these scans can be implemented with `nmap`. An equivalent tool for windows is given by `superscan`, and `netcat`.

Countermeasures for port scanning? As we've said, IDSs. Remember that some of the tools will handle packet fragmentation to perform their functionality, and there are newer versions of IDS like `snort` which handles this normally, or fragments could not be interpreted as malicious on their own.

We also have firewalls, detecting SYN scans and ignoring FIN scans. There are also fake attacks that aim to overload logs.

Another important aspect is to establish which operating system is running on the machine. How do we do this? The easiest way is to do banner grabbing. When we interact with an application, even at user level most of the times an application will tell the version it's running, the system, etc. Doing this at TCP level, like with `telnet`, we will also obtain information about ports for example, taking a look at specific protocols run on a specific platform. Another technique is given by stack fingerprinting, looking at implementation details of the TCP/IP protocol stack, which even if it's standardised, it may have some variations, like in the initialisation of fields in packets.

As examples, for windows we typically have services on port 135, 139, etc., which are typically used for windows protocol specifically. In UNIX we might have port 22 (SSH), TCP 111, etc.

The third approach was given by stack fingerprinting. It relies on implementation details which are slightly different depending on the operating system. There are two types:

- passive: looks at TCP/IP fields.
- active: looking at the way the protocol replies. For example if a FIN packet is sent to an open port, for the standard the correct reply should be "no reply",

but there are some implementations in windows which however responds with a FIN-ACK.